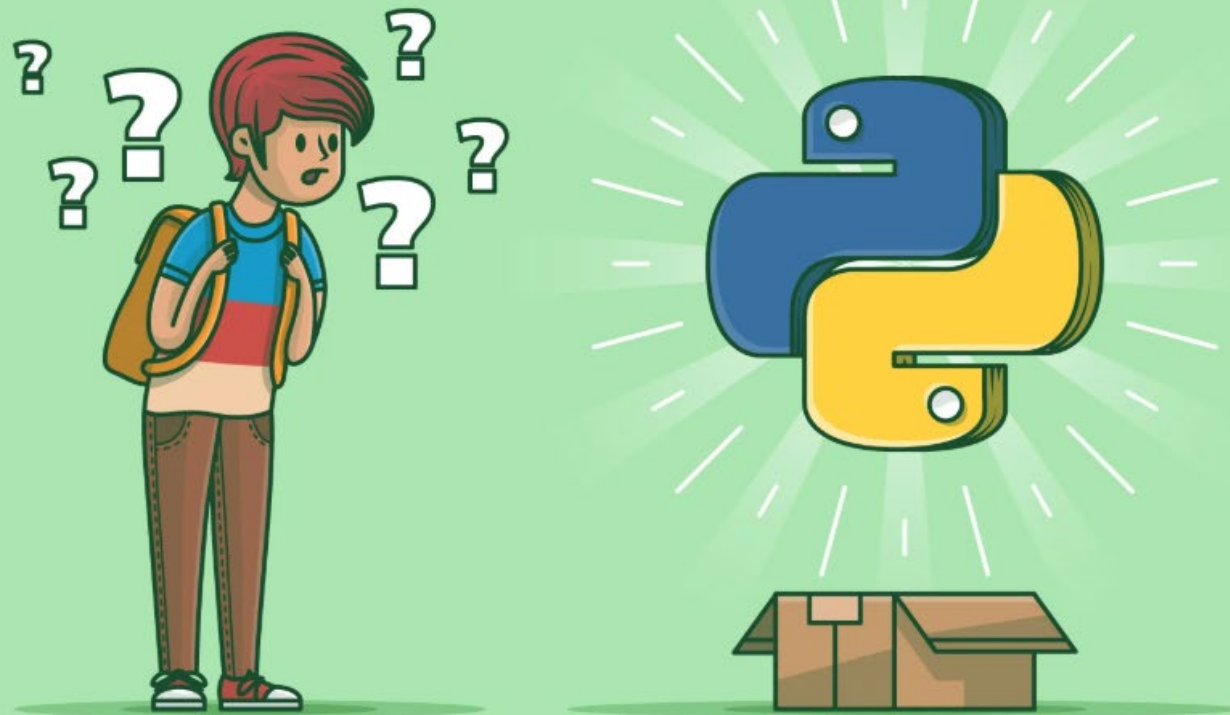


CHAPTER 1 (PART 2) : BASIC OF PYTHON PROGRAMMING



img from <https://realpython.com/>

Real Python

LEARNING OUTCOMES

- Describe and use data types
- Describe and use variables
- Identify Operators
- Identify Operators Precedence
- Use Operators and operators Precedence
- Identify Conditional structure
- Identify Looping structure
- Apply Conditional and Looping structure

Python Syntax

- Its not necessary in Python to end a command with semicolon
- Indentation:
 - refers to the spaces at the beginning of a code line.
 - Where in other programming languages the indentation in code is for readability only, **the indentation in Python is very important.**
 - Python uses indentation to indicate a block of code.

```
if 5 > 2:  
    print("Five is greater than two!")
```

Variable(1)

- Variables are containers for storing data values.
- A variable is created the moment you first assign a value to it.
- Variables **do not need to be declared with any particular type** and can even change type after they have been set.

```
x = 5  
y = "John"  
print(x)  
print(y)
```

Output :

```
5  
John
```

Variable(1)

- The data type of a variable is automatically given to the variable based on the values assign to the variable.

```
1 x = 4 # x is of type int
2 x = "Sally" # x is now of type str
3 print(x)
```

Output :



Variable Name (1)

- Must contain at least one character.
- The first character of a variable must be an alphabetic letter (upper or lower case) or the underscore
- The remaining characters (if any) may be alphabetic characters (upper or lower case), the underscore, or a digit
- No other characters (including spaces) are permitted in identifiers.
- A reserved word cannot be used as an identifier

Variable Name (2)

- Python accepted style
 - my_name, your_name, user_name,
 - account_name, count, total_number_of_users,
 - percentage_passed, pass_rate, where_we_live,
 - house_number, is_okay, is_correct, status_flag

Reserve Word

- The listed keyword cannot be used as variable name

and	del	from	None	try
as	elif	global	nonlocal	True
assert	else	if	not	while
break	except	import	or	with
class	False	in	pass	yield
continue	finally	is	raise	
def	for	lambda	return	

Question 1: Identify whether the following variables is valid or not.

home4you

after_you

\$money

_thatsall

2001

All10

pass

python_101

hello pi

break

Data Type

- Numbers
 - Integers
 - Floating point numbers.
 - Complex number (not covered in this course)
- String
 - strings are defined either with a **single quote** or a **double quotes**.
- Boolean
 - has two values either **True** or **False**
- None
 - **Empty** or **Null** Values

Numbers & Floating Point

Numbers

```
myint = 7  
print(myint)  
print(type(myint))
```

Output :

```
7  
<class 'int'>
```

FLOAT

```
print(type(10.0))  
print(type(10.22))
```

Output :

```
<class 'float'>  
<class 'float'>
```

String(1)

- Strings are defined either with a **single quote** or a **double quotes** or in Python 3 you can use **triple quotes**.
- The difference between **single quote and double quotes** is that it is easy to include apostrophes (example : Aileen's book)
- For **triple quotes**, it allows strings to span multiple lines, including space, NEWLINEs, TABs, and any other special characters
- Any variables created using input() method will be data type string. (To do arithmetic operation you need to convert/cast the variables)

String(2)

Program	Output
<pre>1 name = 'Jane Doe' 2 name2 = "Jane Doe" 3 print(name) 4 print(name2) 5</pre>	<pre>Jane Doe Jane Doe</pre>
<pre>1 mystring = 'Don't worry about apostrophes' 2 print(mystring) 3</pre> <pre>1 mystring = "Don't worry about apostrophes" 2 print(mystring) 3</pre>	<pre>Don't worry about apostrophes</pre>

String(3)

Program	Output
<pre>x = """I'm John Doe, Im studying at PBU. taking Diploma in Information <u>Technoloy</u>, Im in my final year.""" print(x); print(type(x))</pre>	<pre>I'm John Doe, Im studying at PBU. taking Diploma in Information Technoloy, Im in my final year. <class 'str'></pre>
<pre>x = "I'm John Doe,Im studying at PBU \nstudying Diploma in IT" print(x)</pre>	<pre>I'm John Doe,Im studying at PBU studying Diploma in IT</pre>

None Value

- The None keyword is used to define a null value, or no value at all.
- None is **not the same as 0, False**, or an **empty string**.
- None is a datatype of its own (NoneType)

```
1 x = None
2 if x:
3     print("Do you think None is True")
4 else:
5     print("None is not True...")
```

Output :

```
None is not True...
```

Boolean(1)

- Booleans represent one of two values: **True** or **False**.
- You can evaluate any expression in Python, and get one of two answers, True or False.
 - When you compare two values, the expression is evaluated and Python returns the Boolean answer:
 - `print( 10 > 9 )` will return True
 - `print(5<1)` will return False
 - When you run a condition in an if statement, Python returns True or False:

```
a = 200
b = 33

if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

Output :

```
b is not greater than a
```

The `bool()` function(1)

- converts a value to Boolean (True or False) using the standard truth testing procedure.
- `bool()` takes a single parameter value.
- Return False if the value is omitted or false
- Return True if the value is true

The `bool()` function(2)

- The following values are considered false in Python:
 - `None`
 - `False`
 - Zero of any numeric type. For example, `0`, `0.0`, `0j`
 - Empty sequence. For example, `()`, `[]`, `''`.
 - Empty mapping. For example, `{}`
 - objects of Classes which has `__bool__()` or `__len__()` method which returns `0` or `False`
- All other values except these values are considered true.

The bool () function(3)

Output :

```
1 print(bool('a'))  
2 print(bool(True))  
3 print(bool(1))  
4 print(bool(123))  
5 print(bool(0))  
6 print(bool())  
7 print(bool(None))  
8 print(bool(''))
```

```
True  
True  
True  
True  
False  
False  
False  
False
```

Python Comment(1)

- Comments can be used to :
 - explain Python code.
 - make the code more readable.
 - prevent execution when testing code
- Comments starts with a #
- Comments can be placed at the beginning or at the end of a line, and Python will ignore the rest of the line:

```
#This is a comment  
print("Hello, World!")
```

```
print("Hello, World!") #This is a comment
```

Python Comment(2)

- Multi Line Comments

- Python does not really have a syntax for multi line comments.
- To add a multiline comment you could insert a # for each line
- Or, not quite as intended, you can use a multiline string.
- Since Python will ignore string literals that are not assigned to a variable, you can add a multiline string (triple quotes) in your code, and place your comment inside it:

```
#This is a comment  
#written in  
#more than just one line  
print("Hello, World!")
```

```
"""  
This is a comment  
written in  
more than just one line  
"""  
print("Hello, World!")
```

Python Casting

- There may be times when you want to specify a type on to a variable. This can be done with casting.
- Python is an object-orientated language, and as such it uses classes to define data types, including its primitive types.
- Casting in python is therefore done using constructor functions:
 - `int()` - constructs an integer number from an integer literal, a float literal (by rounding down to the previous whole number), or a string literal (providing the string represents a whole number)
 - `float()` - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
 - `str()` - constructs a string from a wide variety of data types, including strings, integer literals and float literals

Python Operators

- Operators are used to perform operations on variables and values.
- Python divides the operators in the following groups:
 - **Arithmetic operators**
 - **Assignment operators**
 - **Comparison operators**
 - **Logical operators**

Arithmetic operators

Expression	Meaning
$x + y$	x added to y , if x and y are numbers x concatenated to y , if x and y are strings
$x - y$	x take away y , if x and y are numbers
$x * y$	x times y , if x and y are numbers x concatenated with itself y times, if x is a string and y is an integer y concatenated with itself x times, if y is a string and x is an integer
x / y	x divided by y , if x and y are numbers
$x // y$	Floor of x divided by y , if x and y are numbers
$x \% y$	Remainder of x divided by y , if x and y are numbers
$x ** y$	x raised to y power, if x and y are numbers

Assignment operators

- Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Comparison operators(1)

- Comparison or relational operators are used to compare two values:

Expression	Meaning
$x == y$	True if $x = y$ (mathematical equality, not assignment); otherwise, false
$x < y$	True if $x < y$; otherwise, false
$x <= y$	True if $x \leq y$; otherwise, false
$x > y$	True if $x > y$; otherwise, false
$x >= y$	True if $x \geq y$; otherwise, false
$x != y$	True if $x \neq y$; otherwise, false

Logical operators

- Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>

Conditional Statement

- Execution on program based on condition.
- In python :
 - If
 - If.. Else
 - Nested if
 - elif

Note that the indentation in Python is very important!.

If statement

Program	Output
<pre>1 num = int(input('Enter a number: ')) 2 if num < 0: 3 print(num, 'is negative')</pre> 	<pre>Enter a number :-10 -10 is negative</pre>
<pre>1 num = int(input('Enter a number: ')) 2 if num < 0: 3 print(num, 'is negative')</pre> 	<pre>File "C:/Users/JTMK-Liyana/PycharmProjects/covid/notes.py", line 3 print(num, 'is negative') ^ IndentationError: expected an indented block</pre>

If.. Else statement

```
1 num = int(input("Enter a number :"))
2 if (num < 0):
3     print(num, "is negative")
4 else:
5     print("the number not negative")
```

Output :

```
Enter a number :-1
-1 is negative
```

Output :

```
Enter a number :10
the number not negative
```

The elif statement

- This else-if scenario is supported in Python by the elif element of an if statement.

```
1 savings = float(input("Enter how much "
2                        " have in savings: "))
3 if savings == 0:
4     print("Sorry no savings")
5 elif savings < 500:
6     print('Well done')
7 elif savings < 1000:
8     print('Thats a tidy sum')
9 elif savings < 10000:
10    print('Welcome Sir!')
11 else:
12    print('Thank you')
```

Output :

```
Enter how much you have in savings: 500
Thats a tidy sum
```

Output :

```
Enter how much have in savings: 0
Sorry no savings
```

Nested if... else

```
1 value = int(input("Please enter an "  
2     "integer value in "  
3     "the range 0...10: "))  
4 if value >= 0: # First check  
5     if value <= 10: # Second check  
6         print(value, "is in range")  
7     else:  
8         print(value, "is too large")  
9 else:  
10     print(value, "is too small")  
11 print("Done")
```

Output :

```
Please enter an integer value in the range 0...10: 20  
20 is too large  
Done
```

Output :

```
Please enter an integer value in the range 0...10: 9  
9 is in range  
Done
```

Output :

```
Please enter an integer value in the range 0...10: -1  
-1 is too small  
Done
```

Looping

- Iteration(looping) repeats the execution of a sequence of code.
- Iteration is useful for solving many programming problems. Iteration and conditional execution form the basis for algorithm construction.\
- Python has two primitive loop command which are:
 - While Loop
 - For Loop
- Other statement that can be use in looping
 - Break - stop the loop even if the while condition is true
 - Continue - stop the current iteration, and continue with the next

while statement(1)

- With the while loop we can execute a set of statements as long as a condition is true.

Syntax :

```
while <test-condition-is-true>:  
    statement or statements
```

```
1 count = 0  
2 print('Starting')  
3 while count < 10:  
4     print(count, ' ', end='') # part of the while loop  
5     count += 1 # also part of the while loop  
6 print()  
7 print('Done')
```

Output :

```
Starting  
0 1 2 3 4 5 6 7 8 9  
Done
```

for loop(1)

- A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).
- To loop through a set of code a specified number of times, we can use the range() function,
- The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.
- Syntax :

```
for val in sequence:  
    Body of for
```

for loop

- Compared to C++, Python for loop has a bit difference in syntax

C++ :

```
for (int i = 1; i <= 10; i++)  
{  
    cout << "Hello World\n";  
}
```

python :

```
1 for i in range(10):  
2     print("Hello World")
```

for loop(2)

Syntax :

```
for <variable-name> in range(...):  
    statement  
    statement
```

INCREMENT BY 1 :

```
1  print('Print out values in a range')  
2  for i in range(0, 10):  
3      print(i, ' ', end='')  
4  print()  
5  print('Done')
```

Output :

```
Print out values in a range  
0 1 2 3 4 5 6 7 8 9  
Done
```

INCREMENT BY 2 :

```
1  print('Print out values in a range with an increment of 2')  
2  for i in range(0, 10, 2):  
3      print(i, ' ', end='')  
4  print()  
5  print('Done')
```

Output :

```
Print out values in a range with an increment of 2  
0 2 4 6 8  
Done
```

break statement

- The break statement, when executed, will terminate the current loop and jump the program to the first line after the loop.

```
1 i = 1
2 while i < 6:
3     print(i)
4     if (i == 3):
5         break
6     i += 1
```

Output :

1
2
3

Continue Statement

- The continue statement skip over part of a loop's iteration for a particular value, but then to continue with the remaining values in the sequence.

```
1 i = 0
2 while i < 6:
3     i += 1
4     if i == 3:
5         continue
6     print(i)
```

Output :

1
2
4
5
6